

AED2 2026 (1s) - AP-04 - ORDENAÇÃO COM HEAPSORT E CONTAGEM

Instruções:

1. E/S: tanto a entrada quanto a saída de dados devem ser "secas", ou seja, sem frases explicativas. Siga o modelo fornecido e apenas complete as partes informadas (veja os exemplos fornecidos);
2. Identificadores de variáveis: escolha nomes apropriados;
3. Documentação: inclua cabeçalho, comentários e indentação no programa;
4. O código-fonte deve ser desenvolvido em linguagem C;
5. Não é permitido o uso da função `Sort(...)` da linguagem C ou o uso de estruturas do tipo `<list>` ou `<vector>`;
6. Tempo limite de solução no sistema Judge: 1 segundo;
7. Submeta o programa no sistema Judge (link igual para ambas as turmas): <https://judge.unifesp.br/AEDII1S2026IN/>;
8. Plágios serão checados pela plataforma MOSS (<https://theory.stanford.edu/~aiken/moss/>). Caso detectados, não serão tolerados, sendo a submissão descartada.
9. Soluções que não sigam as diretrizes acima serão desconsideradas.

Descrição

Nesta atividade, você deve implementar um sistema de ordenação que utiliza dois conceitos fundamentais: um filtro de densidade baseado em frequência (fase linear) e o algoritmo *HeapSort* para a organização final.

Regras do Jogo:

1. **Fase de Contagem Global ($O(n)$):** Antes de validar ou ordenar, o programa deve ler todas as N palavras e contar a frequência absoluta de cada caractere no conjunto.
2. **Fase de Filtragem (K-Densidade):** Uma palavra é considerada instável se contiver qualquer caractere que apareça menos de K vezes no texto fornecido.
 - Palavras instáveis devem disparar a mensagem:
A palavra [xxx] eh invalida (K=[valor]).
 - Estas palavras devem ser **removidas** do conjunto e não participam da ordenação por *HeapSort*.

3. **Fase de Ordenação (*HeapSort*):** As M palavras restantes devem ser ordenadas.
- **Chave de Ordenação:** A soma das frequências (calculadas na Fase 1) de cada letra da palavra.
 - **Desempate:** Caso os pesos sejam iguais, utilize a ordem alfabética inversa (Z para A).

Restrições de Implementação (Obrigatório)

1. A fase de contagem de frequência de letras deve ter **complexidade** $O(N)$, onde N representa a quantidade de palavras do conjunto de entrada.
2. **Adaptação do Algoritmo da disciplina:** O código-fonte para o algoritmo de ordenação deve ser uma adaptação direta do algoritmo *HeapSort* visto em sala de aula, **devendo ter complexidade** $O(M \log_2 M)$, onde M representa a quantidade de palavras válidas a serem ordenadas ($M \leq N$).
3. **Memória:** Toda memória alocada dinamicamente deve ser liberada. Proibido o uso de variáveis globais.
4. **Algoritmo *In-Place*:** A ordenação deve ser realizada *in-place* (no próprio local), para minimizar o uso de memória. É proibida a criação de vetores auxiliares ou cópias parciais do vetor original durante as etapas de partição. Toda a manipulação deve ser feita via trocas (*swaps*) de elementos dentro do único vetor alocado no início do programa.
5. Todas as palavras (*strings*) do conjunto de entrada estarão em **caracteres minúsculos**.
6. Cada palavra da entrada será composta exclusivamente por letras minúsculas do alfabeto inglês (a até z). Não haverá a presença de números, símbolos, caracteres especiais ou letras acentuadas. O tamanho máximo de cada palavra é de 20 caracteres.

Entrada

- A primeira linha contém dois inteiros N ($1 \leq N \leq 1.000$) e k ($1 \leq k \leq 10$), representando o número total de palavras do conjunto de entrada e a frequência mínima exigida para cada caractere (conforme as regras do jogo), respectivamente.
- Na seguinte linha 2, tem-se o conjunto de entrada, composto de N palavras (*strings*), separadas por um caractere de espaço em branco.

Saída

- **Avisos de Invalidação:** Para cada palavra instável detectada (na ordem em que aparecem na entrada), imprimir: A palavra [xxx] eh invalida (K=[valor])
- **Estado do Heap:** Caso existam palavras válidas, imprimir a frase: `string build_heap:` seguida pelo estado do *heap* (conjunto de palavras que compõem o *heap*) imediatamente após a construção do *heap* inicial.

- **Resultado Final:** Imprimir a *string* palavras: seguida pelas M palavras ordenadas, separadas por um único espaço.
- **Exceção:** Caso nenhuma palavra reste após a filtragem, imprimir:
 Erro: Sem palavras validas para ordenacao.

Exemplos de Caso de Teste

Exemplo 1: Todas as palavras válidas ($K = 1$)

Entrada:

```
3 1
abc bca acc
```

Saída:

```
build_heap: acc bca abc
palavras: abc bca acc
```

Explicação:

1. Contagem de frequência de letras: a:3, b:2, c:4.
2. Pesos:
 - abc: $3 + 2 + 4 = 9$
 - bca: $2 + 3 + 4 = 9$
 - acc: $3 + 4 + 4 = 11$
3. *Heap*: acc tem o maior peso (11), ficando na raiz. No empate entre abc e bca (ambos 9), bca é considerado "maior" no *heap* por vir depois alfabeticamente (regra de desempate inverso). Por isso, bca aparece antes de abc no build_heap.

Exemplo 2: Filtragem de palavra instável ($K = 2$)

Entrada:

```
4 2
banana batata abacaxi dende
```

Saída:

```
A palavra abacaxi eh invalida (K=2)
build_heap: banana batata dende
palavras: dende batata banana
```

Explicação:

1. Contagem Global: Ao somar todas as letras, nota-se que x e i aparecem apenas 1 vez (na palavra abacaxi).
2. Filtro: Como $K = 2$ a palavra abacaxi é descartada por conter caracteres com frequência $< K$.

3. Ordenação: Apenas banana, batata e dende sobram. O *HeapSort* organiza essas três com base em seus pesos de frequência global. Note que dende tem o menor peso total de frequências, terminando na primeira posição da lista final (ordem crescente de peso).

Exemplo 3: Conjunto totalmente inválido (K alto)

Entrada:

```
2 5
raro comum
```

Saída:

```
A palavra raro eh invalida (K=5)
A palavra comum eh invalida (K=5)
Erro: Sem palavras validas para ordenacao.
```

Explicação:

1. Contagem: Nenhuma letra (r, a, o, c, m, u, etc.) aparece 5 vezes no total.
2. Filtro: A primeira palavra raro é validada e falha. A segunda palavra comum também falha.
3. Finalização: O programa detecta que o número de palavras válidas (M) é zero e emite a mensagem de erro específica em vez de tentar construir um *heap* vazio.